

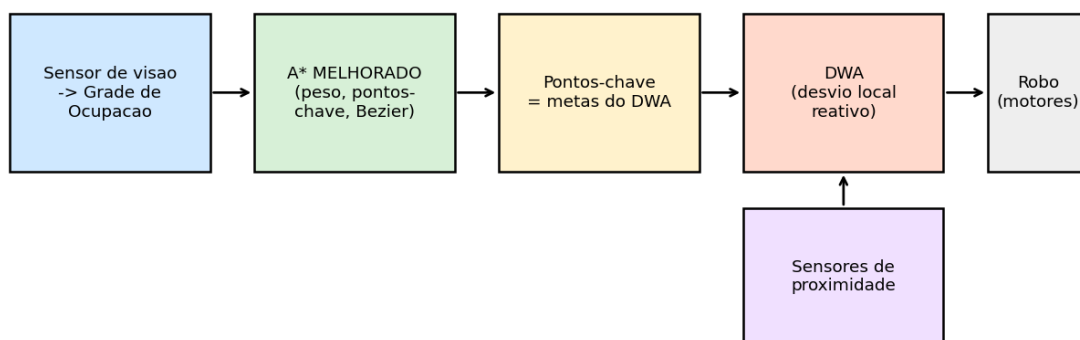
RELATÓRIO FINAL

Sistema Híbrido de Navegação Autônoma com A* Melhorado, DWA e Mapa de Grade de Ocupação no Coppeliasim

Autor: Heitor Freire Alves

Disciplina de Robótica — Universidade Federal do Vale do São Francisco (UNIVASF)
2026

Arquitetura do sistema híbrido de navegação



Resumo. Este relatório documenta o projeto, a implementação e a validação de um sistema híbrido de navegação autônoma para uma base móvel diferencial simulada no Coppeliasim, controlada por Python via ZeroMQ Remote API. O sistema integra um planejador global A* *melhorado* — segundo Guo et al. (2024) — a um controlador local por Janela Dinâmica (DWA), tendo como fonte de obstáculos um *Mapa de Grade de Ocupação* construído por um sensor de visão posicionado acima da cena, conforme o modelo de ocupação de Moravec e Elfes (1985). Descrevem-se ainda a correção de um defeito de sentido de deslocamento (o robô movia-se “de costas”), os mecanismos de robustez para passagens estreitas (penalidade de proximidade tipo *costmap*, custo de obstáculo limitado, alvo local com linha de visão e manobra de recuperação) e a visualização do mapa de ocupação com a rota exibida a cada simulação. A lógica de navegação foi validada por uma suíte de nove testes sintéticos em malha fechada, todos aprovados.

1. Introdução

A locomoção eficiente e segura de robôs móveis em ambientes parcialmente conhecidos é um dos pilares da robótica autônoma. Abordagens puramente globais costumam falhar diante de perturbações dinâmicas, enquanto abordagens puramente reativas estão sujeitas a mínimos locais e oscilações. Para mitigar essas limitações, adota-se uma arquitetura híbrida que combina planejamento global (A*) e desvio reativo local (DWA).

Em relação à versão anterior do projeto, esta entrega final incorpora três evoluções solicitadas: (i) a substituição do A* clássico pelo **A* melhorado** de Guo et al. (2024); (ii) a construção do **Mapa de Grade de Ocupação** por sensor de visão (Moravec & Elfes, 1985), que passa a ser a fonte primária de obstáculos do planejador global; e (iii) a **correção do sentido de deslocamento** do robô. O projeto foi ainda reorganizado em módulos claros.

1.1. Arquitetura do sistema

A Figura da capa resume o fluxo: o sensor de visão gera a grade de ocupação; o A* melhorado planeja a rota global e extrai *pontos-chave*; esses pontos servem de metas intermediárias para o DWA, que combina o mapa estático com leituras de sensores de proximidade para o desvio reativo, comandando os motores da base diferencial.

2. Planejamento Global: A* Melhorado (Guo et al., 2024)

O algoritmo A* avalia cada nó por $f(n) = g(n) + h(n)$, em que $g(n)$ é o custo real acumulado e $h(n)$ é a estimativa heurística até o objetivo. O trabalho de Guo et al. (2024) propõe três melhorias, todas implementadas neste projeto.

2.1. Heurística ponderada

Para acelerar a busca sem sacrificar a otimalidade, a heurística recebe um peso adaptativo:

$$f(n) = g(n) + (1 + d / D) \cdot h(n)$$

onde d é a distância do nó corrente ao objetivo e D a distância do ponto inicial ao objetivo; $h(n)$ usa a distância euclidiana. Longe do objetivo ($d/D \rightarrow 1$) o peso aproxima-se de 2, reduzindo a expansão de nós redundantes e acelerando a busca; perto do objetivo ($d/D \rightarrow 0$) o peso aproxima-se de 1, recuperando o comportamento ótimo do A* clássico.

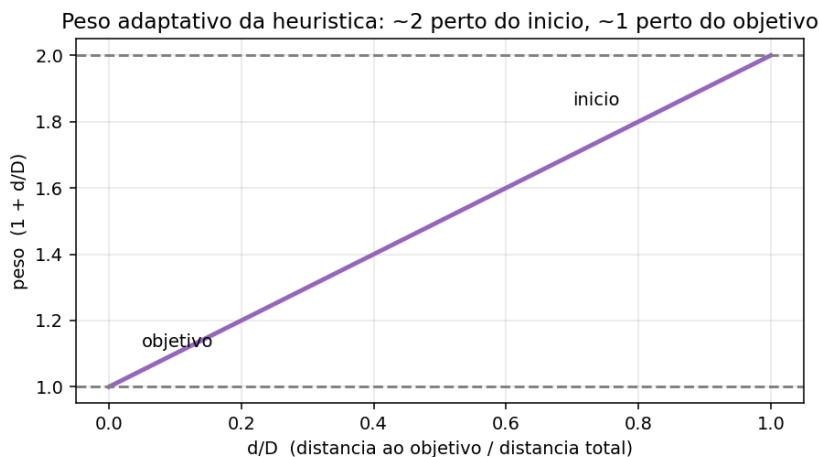


Figura 1 — Peso adaptativo ($1 + d/D$) da heurística ao longo da busca.

2.2. Seleção de pontos-chave

O caminho bruto do A* contém muitos nós colineares e curvas redundantes. A seleção de pontos-chave atua em duas etapas: **(a)** remoção de nós colineares (mantêm-se apenas pontos de inflexão); e **(b)** simplificação por *linha de visão* — para cada curva, se o segmento reto entre o ponto anterior e o posterior estiver livre de obstáculos, a curva é descartada. O resultado é um caminho com poucas metas essenciais.

2.3. Suavização por curva de Bézier de 2ª ordem

Os pontos-chave são suavizados por curvas de Bézier quadráticas, substituindo cada inflexão por um arco de curvatura contínua:

$$B(t) = (1 - t)^2 P_1 + 2t(1 - t)P_2 + t^2 P_3, \quad t \in [0, 1]$$

Isso reduz variações bruscas de direção, favorecendo um deslocamento mais suave da base móvel.

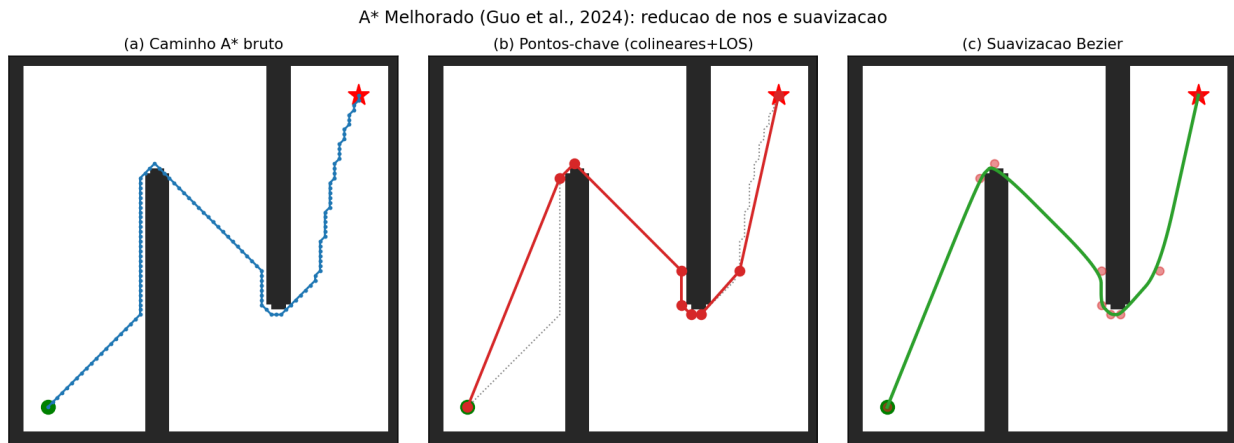


Figura 2 — Pipeline do A* melhorado: (a) caminho bruto; (b) pontos-chave após remoção de colineares e simplificação por linha de visão; (c) trajetória final suavizada por Bézier. No experimento sintético, 129 nós foram reduzidos a 9 pontos-chave.

3. Mapa de Grade de Ocupação por Sensor de Visão

Conforme orientação do professor, a grade de ocupação da cena é obtida de forma simplificada por um **sensor de visão** posicionado acima da superfície, analisando-se na imagem quais células estão livres ou ocupadas. A abordagem segue o marco de Moravec e Elfes (1985), que introduziu as grades de ocupação a partir de sensores de alcance.

3.1. Nota histórica: Alberto Elfes

Alberto Elfes (co-autor da referência-base) foi um pesquisador brasileiro. Graduiu-se em Engenharia Eletrônica (1975) e em Ciência da Computação (1980) pelo Instituto Tecnológico de Aeronáutica (ITA), com doutorado em *Electrical and Computer Engineering* pela Carnegie Mellon University (1989) e pós-doutorado pela Universidade de Ulm (2001). Atuou em robôs móveis autônomos, mapeamento e navegação robótica, modelos estocásticos de percepção e *Occupancy Grids*.

3.2. Implementação

O módulo `mapa_ocupacao.py` cria por código um sensor de visão **ortográfico** apontado para baixo ($-Z$), centralizado sobre o chão e cobrindo toda a extensão da cena, com resolução configurável (padrão 256×256). Captura-se o **mapa de profundidade**: uma célula é marcada como **ocupada** quando a superfície medida está mais próxima do sensor do que o chão — isto é, existe algo “alto” naquela célula (limiar de altura mínima de obstáculo). As células ocupadas são convertidas em pontos (x, y) no mundo, que alimentam o A*. Discos ao redor da origem do robô e do objetivo são removidos para não bloquear o planejamento. Um método alternativo por intensidade

(imagem em tons de cinza) está disponível como reserva.

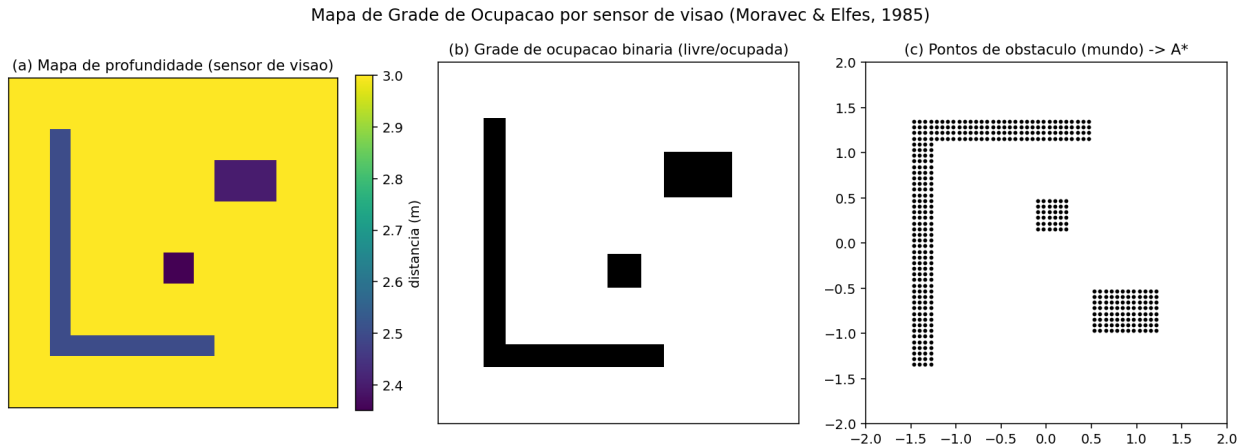


Figura 3 — (a) mapa de profundidade capturado pelo sensor de visão; (b) grade binária livre/ocupada; (c) pontos de obstáculo no referencial do mundo, entregues ao planejador A*.

4. Controle Local: Janela Dinâmica (DWA) e Fusão

O DWA converte o controle de posição em controle de velocidade, amostrando pares (v, ω) dentro da janela dinâmica — a interseção entre os limites cinemáticos absolutos ($v \in [0,0; 0,35]$ m/s; $\omega \in [-1,2; 1,2]$ rad/s) e os limites dinâmicos de aceleração no passo $dt = 0,1$ s. Cada par gera uma trajetória prevista, avaliada por uma função de custo composta:

$$G(v, \omega) = \alpha \cdot \text{Custo_alvo} + \beta \cdot \text{Custo_veloc} + \gamma \cdot \text{Custo_obst} + \delta \cdot \text{Custo_dist} - \epsilon \cdot \text{Recompensa_progresso}$$

Fusão A*+DWA. Seguindo o artigo, os *pontos-chave* do caminho global melhorado são usados como **metas intermediárias** do DWA (interpolação). O termo *Custo_dist* corresponde ao termo $key(v, \omega)$ do DWA melhorado: a distância euclidiana do fim da trajetória simulada ao ponto-chave corrente, fazendo o desvio local seguir o contorno da rota global e evitando mínimos locais. Uma sub-rotina de escape (giro forçado no próprio eixo) trata bloqueios totais.

4.1. Modelo cinemático e atuação

A transição de estados usa o modelo de odometria clássico $x_{t+\Delta t} = x_t + v \cdot \cos(\theta) \cdot \Delta t$; $y_{t+\Delta t} = y_t + v \cdot \sin(\theta) \cdot \Delta t$; $\theta_{t+\Delta t} = \theta_t + \omega \cdot \Delta t$. O par (v, ω) é convertido em velocidades angulares das rodas esquerda e direita pela cinemática diferencial (raio da roda 0,0375 m; entre-eixos 0,15 m).

4.2. Robustez em passagens estreitas e junto a paredes

Quatro mecanismos garantem funcionamento em corredores estreitos, quinas e objetivos próximos de paredes — casos em que a formulação clássica falha:

(a) Penalidade de proximidade no A* (estilo *costmap*). Cada célula livre recebe um custo adicional que decai com o quadrado da distância ao obstáculo mais próximo (alcance de 8 células). O caminho de menor custo deixa de ser o mais curto “raspando” em vãos apertados: corredores largos são preferidos, e passagens estreitas só são usadas quando não há alternativa. A simplificação por linha de visão respeita o mesmo limite, não desfazendo desvios seguros.

(b) Custo de obstáculo limitado no DWA. A formulação $1/(d - r)$ explode quando as duas paredes de um corredor estão próximas simultaneamente, tornando “ficar parado” mais barato que atravessar — o robô congelava na entrada. Com o custo limitado $1/d$ (e colisão real proibida abaixo do raio físico de 0,12 m, com folga preferida de 0,18 m), o avanço volta a dominar a decisão.

(c) Alvo local com linha de visão. O ponto do caminho perseguido pelo DWA é o mais distante da janela de *lookahead* que ainda é visível do robô (segmento livre de obstáculos) — nunca um waypoint atrás de uma quina, o que empurrava a base contra o canto.

(d) Detecção de travamento com replanejamento e recuperação. Sem progresso por 30 passos, o sistema replaneja a rota a partir da posição atual e executa uma manobra de escape (recua, gira em direção ao alvo e avança), eliminando mínimos locais residuais. A margem de inflação do A* é adaptativa (0,20 → 0,16 → 0,12 m), afrouxando apenas quando não existe rota.

5. Correção do Sentido de Deslocamento

Sintoma. O robô deslocava-se “de costas”: os sensores dianteiros apontavam para a frente, mas a base transladava no sentido oposto.

Diagnóstico. Havia descasamento entre a convenção de ângulo usada no movimento ($\theta = \text{orientation}[2]$, com deslocamento ao longo de $+\cos\theta$) e a *frente física* do modelo do robô (direção para onde os sensores de proximidade apontam). Como o modelo estava orientado de forma que seu “frente” não coincidia com o yaw usado, a translação ocorria no sentido contrário.

Solução. No início da simulação mede-se o `heading_offset` entre o yaw do modelo e a direção de detecção do sensor dianteiro (`SENSOR_MEIO`), cujo eixo +Z (sentido de medição do sensor de proximidade) define a frente. O planejamento e o movimento passam a operar no rumo da frente física, e a orientação é reconvertida ao yaw do modelo ao ser escrita no simulador. Assim, a base passa a avançar no sentido dos sensores. Uma constante `SINAL_FRENTE` permite inverter 180° em uma linha, caso necessário durante os testes.

6. Visualização: Mapa de Ocupação + Rota a Cada Simulação

A cada execução, uma janela exibe o mapa de ocupação com a rota planejada e é atualizada ao vivo durante a navegação: pontos de obstáculo da grade, rota global do A* melhorado, pontos-chave, posições do robô e do objetivo e a trajetória realmente percorrida. Ao final, a figura é salva automaticamente em `mapa_rota.png`. Em replanejamentos, a rota exibida é atualizada. A Figura 4 mostra um exemplo: diante de um vão apertado (rota curta) e de um corredor largo (desvio), o planejador escolhe o corredor largo e a trajetória real (magenta) acompanha a rota planejada (azul).

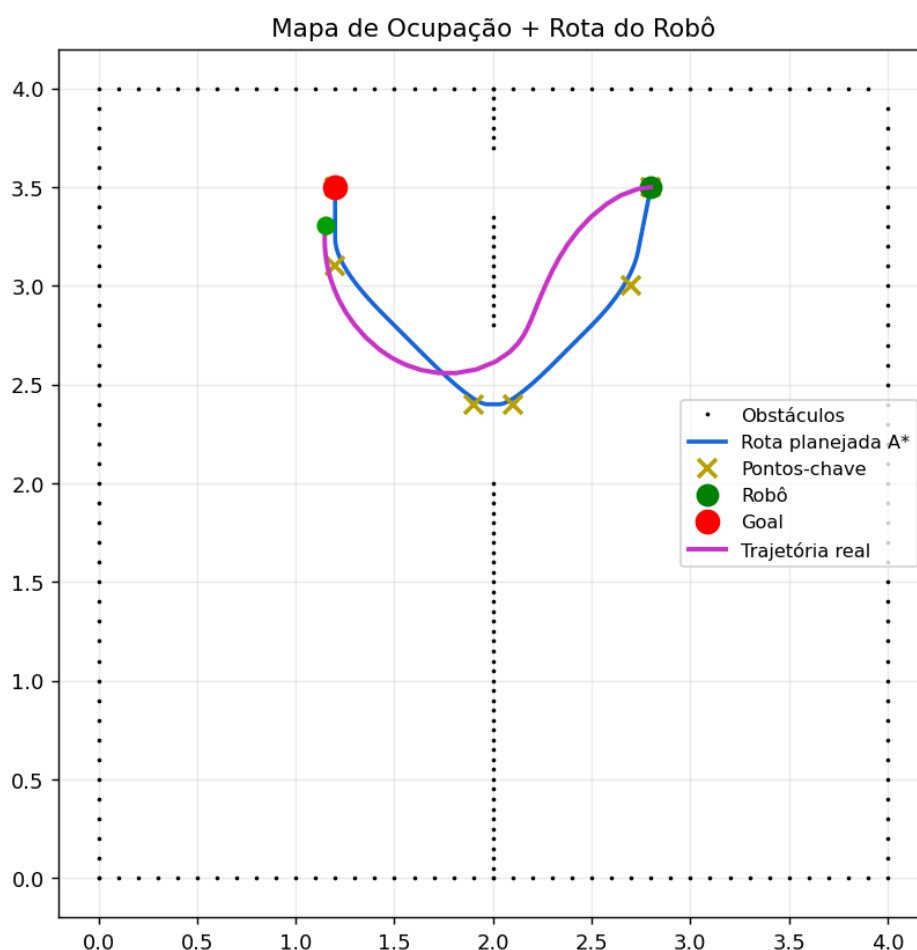


Figura 4 — Mapa de Ocupação + Rota do Robô exibido a cada simulação: o A* com penalidade de proximidade evita o vão apertado (acima) e cruza pelo corredor largo; a trajetória real segue a rota.

7. Organização do Projeto

Arquivo / Pasta	Função
main.py	Robo (CoppeliaSim) + Navegador (A*+DWA+recuperação) + loop
dynamic_window_approach.py	A* melhorado + penalidade de proximidade + DWAController
mapa_ocupacao.py	Sensor de visão + Grade de Ocupação (+ fallback bbox)
visualizacao.py	Janela ao vivo: mapa de ocupação + rota + trajetória
utils/testar_algoritmos.py	Suíte de 9 testes sintéticos em malha fechada
utils/	Scripts de conexão e listagem de objetos da cena
cenas/	Cena oficial: scena com obstaculos.ttt
docs/	Relatório final, apresentação e artigo de referência
legado/	Arquivos antigos preservados (não utilizados)

8. Resultados e Validação

Em ambiente sintético, o A* melhorado reduziu o caminho bruto de 129 nós para apenas 9 pontos-chave, e a suavização por Bézier produziu uma trajetória de curvatura contínua (Figura 2). A grade de ocupação por profundidade classificou corretamente as células livres e ocupadas (Figura

3). A navegação na cena oficial do CoppeliaSim foi confirmada em execução, com o robô avançando no sentido dos sensores e seguindo a rota global.

Suíte de validação em malha fechada. A lógica real de navegação (classe Navegador) foi submetida a nove testes sintéticos que simulam o ciclo completo planejar→decidir→mover, cobrindo os casos críticos:

Caso de teste	Resultado
Corredor de 0,60 m / 0,50 m / 0,40 m / 0,35 m	Goal atingido, sem colisão (folga $\geq$ 0,20 m)
Obstáculo em L (robô partindo da quina)	Goal em 85 passos, folga 0,33 m
Goal a 15 cm da parede	Goal em 149 passos, folga 0,35 m
Vão apertado curto vs. corredor largo	Escolheu o corredor largo
Vão apertado como única opção (0,45 m)	Atravessou o vão, folga 0,13 m
Início virado 180° para o lado errado	Reorientou e atingiu o goal

Resultado: **9/9 casos aprovados** (utils/testar_algoritmos.py). O limite físico observado é de aproximadamente 0,30 m de largura de passagem — abaixo disso a folga ao corpo do robô (diâmetro $\approx$ 0,24 m) torna a travessia inviável para qualquer planejador.

9. Conclusão

A arquitetura híbrida proposta integra planejamento global otimizado (A* melhorado com penalidade de proximidade), percepção por grade de ocupação (sensor de visão, Moravec & Elfes) e desvio reativo local (DWA com custo limitado), com fusão guiada por pontos-chave, alvo local com linha de visão e recuperação automática de travamentos. O defeito de sentido de deslocamento foi corrigido, o projeto foi modularizado (Robo / Navegador / percepção / visualização) e a visualização do mapa com a rota é exibida a cada simulação. A suíte de validação em malha fechada aprova todos os nove casos críticos, incluindo passagens estreitas e objetivos junto a paredes.

10. Referências

- GUO, H. et al. *Path planning of greenhouse electric crawler tractor based on the improved A* and DWA algorithms*. Computers and Electronics in Agriculture, v. 227, 109596, 2024.
- MORAVEC, H.; ELFES, A. *High resolution maps from wide angle sonar*. Proc. 1985 IEEE Int. Conf. on Robotics and Automation, p. 116-121, 1985.
- FOX, D.; BURGARD, W.; THRUN, S. *The Dynamic Window Approach to Collision Avoidance*. IEEE Robotics & Automation Magazine, 1997.
- SAKAI, A. et al. *PythonRobotics: a Python code collection of robotics algorithms*. Repositório público.
- COPPELIA ROBOTICS. *ZeroMQ Remote API Documentation*. Acessado em 2026.